AI Coder

Neural Sketch Based Framework

Bhagvan Kommadi
Quantica Computacao

Hyderabad, India
bhagvank@quanticacomputacao.com

***Abstract*— To start with, AI Coder is about an AI Bot which can generate code in PHP, Ruby on Rails, Django, Java, Python-based AI Coder can generate code in python and Django or any other frameworks. Typically, we provide details regarding the software and how it works. This takes effort which is equal to writing the code. AI Coder needs basic info about the entities involved in the code. It can train itself by reading zillions of human-written code. It can read from a repository and generate code based on the components. It is useful for creating samples of code and unit tests for software frameworks. Data generation for testing is another feature of AI Coder. AI Coder can take input as rules, tools, definitions, protocols, database support and or other software support. It can be enhanced to support developed software API for code generation. AI Coder is based on neural sketch learning. Neural sketch learning is related to an ANN trained to identify the code level patterns from different software.**

Keywords*— *neural, native, hybrid, app, application (key words)

I. INTRODUCTION

In code generation, generated code needs to be based on the syntax and semantics configured in the input specification. The mobile application framework which is based on the neural sketch learns by training. Training is done by displaying the applications which are configured in the framework. The framework model will have the program specification which is the class, methods and the entities related to the application. The mobile application framework renders the application by using the knowledge gained over the training. The application will have the application programming interface calls, methods and data types based on the entities and the input configuration.

Neural sketch based mobile application framework mixes the neural and combinatorial methods for displaying and rendering the native, hybrid and web based mobile applications. This can be used for iOS, Android and Windows operating system based devices and different programming language sdks supported by the devices.

Supervised learning is the method used for training the mobile application framework. The training input has set of applications which are configured based on the views, entities and actions. The metadata provided by the application configuration helps the framework to learn the semantics of the application. The application has data types, methods, control flow, exceptions and type-based entities.

The patterns are identified by the neural sketch-based framework which helps in rendering and displaying the application. Application needs to satisfy the constraints specified in the configuration file. The neural sketch framework is based on the mix of neural learning and combinatorial search. The source code in the framework has capability to create tree based syntactic models and sketches.

II. NEURAL SKETCH - APPLICATION FRAMEWORK

A sketch is related to a class, entity and methods. The methods have arguments, return values and control flows. The architectural approach is based on the metadata driven architecture pattern. The metadata is stored and modeled using the object repository pattern. The code reads the metadata and renders the mobile application assuming microservices based architecture. The architecture has service layer, services client (mobile), persistence layer, data access layer and data base layers. These layers are implemented using language sdk and platform specific patterns and practices. The meta data attributes are configured in the input for different layers. The application is configurable and extensible with events, delegates, templates, services and classes.

The code in the framework uses the metadata for handling different layers in the language sdk and platform. The help documentation is also configured, and it is used for application rendering and displaying. The user interface can be complex and will need layouts and additional configuration for different UI controls. The complex business logic will be passed on through custom modules and classes. The framework has capabilities to handle upgrades, versions and maintenance related features.

The neural sketch approach is based on latest technology and can be upgraded to any new technology stack by changing the application framework code stack in different layers. The database layer persists with the data. It also handles the current state. The future scheduled state modifications can be handled by the database layer. Persistence layer has features to store and retrieve the data from various data sources. The retrieval of the data is done by using different adapters. The business logic and the rules layer handle the complex changes across the business entities and the entity groups. The service layer handles the updates and managing the group of entities. The user interface layer has features related to customization, configurability, extensibility of the attributes and entities.

Different organization use metadata driven approach for creating the mobile application framework-based products. (e.g., Oracle). Metadata is used to create the application by

dynamic patterns and generation of the presentation layer. The other layers are modeled in the input configuration files. The modeling is based on the generic attribute-based entity patterns.

The application framework helps with upgrades, customizations, maintenance and other challenges in software engineering lifecycles. The user interface is provided for handling customizations. These customizations do not require any code changes. The custom modules can be coded, which can be loaded by the application framework.

The user interface for the applications will be typically simple and easy to use. The application will be rendered in different languages, format, and organization/layout based on the user's location and setup. Application will have support for globalization, which includes internationalization and localization. The application will have rapid and flexible searching and reporting. Collaboration and Data Sharing will be part of the application features. The application will support data sharing with different non-product users, inside and outside of the system.

III. APPLICATION ARCHITECTURE

The application architecture will have the ability to extend by using commercial packages and software. and use only the functionality needed. Security and licensing requirements will be handled by the application architecture. The architecture handles auditing of modifications such as inserts, edits, and deletions. The application will have configurable, multi-tiered, entity-context sensitive, security access to data and actions for different user roles and groups.

Application configuration will provide the ability to customize certain aspects of the platform like screens, data fields, workflows, etc. The application will have development related configuration related to managing metadata and programmatic updates.

Admin level managing metadata through screens

Search- The platform should allow for full Google-like search functionality for data on the platform

Developer Configuration of managing metadata and programmatic updates

Admin level managing metadata through screens

Document Management - System should be able to support core document management features along with integration with third-party document management software

Business Rules – Platform should provide functionality to define business rules. Business rules can be linked to events or time.

Workflow - The base workflow functionality will be provided by the platform. Workflows specific to a product will be built on top of this workflow functionality.

Scheduler – Platform should allow for admin users to schedule periodic actions

Calendar Functionality – The platform will support scheduling calendar events

Notifications – The platform should be able to send notifications in the form of emails or other formats based on certain rules

Reporting – Platform should support scheduled and adhoc reporting of all available data

Internationalization – Platform will provide support for multiple languages, currencies and locales

Integrations – Provide module which can create event or batch driven synchronization and posting integrations with external systems for supported protocols via a configurable interface

User Interface – Provide a web-based user interface that supports rich application functionality and can be customized by an administrator.

Developer Configuration of managing metadata and programmatic updates

Admin level managing metadata through screens

Domain Objects – The ability to create new domain objects or modify existing domain objects by adding attributes and defining their relationships to other domain objects and reference data.

Developer Configuration of managing metadata and programmatic updates

Admin level managing metadata through screens

The sample configuration is provided which has the form, form fields, views, and entities used in the system.

```
<?xml version="1.0" encoding="UTF-8"?>
<app>
  <view>
    <name>User</name>
    <viewno>1</viewno>
    <viewFields>
      <viewField>UserId</viewField>
      <viewField>UserName</viewField>
        <viewField>Password</viewField>
    </viewFields>
  </view>
```

The application will have a user view. User view will have the view fields which are UserId, UserName and Password.

```
  <form>
    <name>User</name>
```

```
<formno>1</formno>
<formFields>
<formField>UserName</formField>
<formField>Password</formField>
</formFields>
</form>
```

The form configuration for User view will have form fields UserName and Password.

```
<update>
<name>User</name>
<formno>1</formno>
<formIdField>UserId</formIdField>
<searchFormField>UserName</searchFormField>
<formFields>
<formField>UserName</formField>
<formField>Password</formField>
</formFields>
</update>
```

The update view will have formno, formIdField, searchFormField and the form fields UserName and Password.

```
<delete>
<name>User</name>
<formno>1</formno>
<formidField>UserId</formidField>
<searchFormField>UserName</searchFormField>
<formFields>
<formField>UserName</formField>
<formField>Password</formField>
</formFields>
</delete>
</app>
```

The delete view will have formno, formIdField, searchFormField and the form fields UserName and Password.

The configuration below shows the web service request and response schemas. User service will have getAll, getUser, InsertUser, UpdateUser and DeleteUser methods.

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions
targetNamespace="http://businessservices.telecom.teal.archcorner.org" xmlns:apachesoap="http://xml.apache.org/xml-soap"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
xmlns:wsdlsoap="http://schemas.xmlsoap.org/wsdl/soap/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema" >
<wsdl:types>
<schema elementFormDefault="qualified"
targetNamespace="http://businessservices.telecom.teal.archcorner.org" xmlns="http://www.w3.org/2001/XMLSchema">
<import
namespace="http://pojo.telecom.teal.archcorner.org"/>
<element name="getAll">
<complexType/>
</element>
<element name="getAllResponse">
<complexType>
<sequence>
<element maxOccurs="unbounded"
name="getAllReturn" type="tns1:User"/>
</sequence>
</complexType>
</element>
```

Get All method request and response definition is shown above.

```
<element name="getUser">
<complexType>
<sequence>
<element name="userName" type="xsd:string"/>
</sequence>
</complexType>
</element>
<element name="getUserResponse">
<complexType>
<sequence>
<element name="getUserReturn" type="tns1:User"/>
</sequence>
</complexType>
</element>
```

Get User method request and response definition is shown above.

```
<element name="updateUser">
 <complexType>
  <sequence>
   <element name="userProperties" type="xsd:string"/>
  </sequence>
 </complexType>
</element>
<element name="updateUserResponse">
 <complexType/>
</element>
```

updateUser method request and response definition is shown above.

```
 <element name="insertUser">
 <complexType>
  <sequence>
   <element name="userProperties" type="xsd:string"/>
  </sequence>
 </complexType>
</element>
<element name="insertUserResponse">
 <complexType/>
</element>
```

insertUser method request and response definition is shown above.

```
<element name="deleteUser">
 <complexType>
  <sequence>
   <element name="userProperties" type="xsd:string"/>
  </sequence>
 </complexType>
</element>
<element name="deleteUserResponse">
 <complexType/>
</element>
</schema>
```

deleteUser method request and response definition is shown above.

The schema definition for the User complex type is shown below.

```
<schema elementFormDefault="qualified"
targetNamespace="http://pojo.telecom.teal.archcorner.org"
xmlns="http://www.w3.org/2001/XMLSchema">
 <complexType name="User">
  <sequence>
   <element name="password" nillable="true"
type="xsd:string"/>
   <element name="userId" type="xsd:int"/>
   <element name="userName" nillable="true"
type="xsd:string"/>
  </sequence>
 </complexType>
</schema>
</wsdl:types>
```

User complex Type elements are shown above.

```
<wsdl:message name="deleteUserResponse">
  <wsdl:part element="impl:deleteUserResponse"
name="parameters">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="getAllRequest">
  <wsdl:part element="impl:getAll"
name="parameters">
  </wsdl:part>
</wsdl:message>
```

deleteUser response elements are shown above.

```
<wsdl:message name="updateUserResponse">
  <wsdl:part element="impl:updateUserResponse"
name="parameters">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="updateUserRequest">
  <wsdl:part element="impl:updateUser"
name="parameters">
  </wsdl:part>
```

```
</wsdl:message>
```

updateUser response elements are shown above.

```
<wsdl:message name="insertUserRequest">
<wsdl:part element="impl:insertUser" name="parameters">
</wsdl:part>
</wsdl:message>
```

InsertUser request elements are shown above.

```
<wsdl:message name="deleteUserRequest">
<wsdl:part element="impl:deleteUser" name="parameters">
</wsdl:part>
</wsdl:message>
```

deleteUser request elements are shown above.

```
<wsdl:message name="insertUserResponse">
<wsdl:part element="impl:insertUserResponse" name="parameters">
</wsdl:part>
</wsdl:message>
```

InsertUser response elements are shown above.

```
<wsdl:message name="getUserRequest">
<wsdl:part element="impl:getUser" name="parameters">
</wsdl:part>
</wsdl:message>
```

getUser request elements are shown above.

```
<wsdl:message name="getAllResponse">
<wsdl:part element="impl:getAllResponse" name="parameters">
</wsdl:part>
</wsdl:message>
```

getAll response elements are shown above.

```
<wsdl:message name="getUserByIdResponse">
<wsdl:part element="impl:getUserByIdResponse" name="parameters">
</wsdl:part>
</wsdl:message>
```

getUserById response elements are shown above.

```
<wsdl:message name="getUserResponse">
<wsdl:part element="impl:getUserResponse" name="parameters">
</wsdl:part>
</wsdl:message>
```

getUser response elements are shown above.

```
<wsdl:portType name="UserService">
<wsdl:operation name="getAll">
<wsdl:input message="impl:getAllRequest" name="getAllRequest">
</wsdl:input>
<wsdl:output message="impl:getAllResponse" name="getAllResponse">
</wsdl:output>
</wsdl:operation>
```

User service elements are shown above.

```
<wsdl:operation name="getUser">
<wsdl:input message="impl:getUserRequest" name="getUserRequest">
</wsdl:input>
<wsdl:output message="impl:getUserResponse" name="getUserResponse">
</wsdl:output>
</wsdl:operation>
```

getUser operation input and message elements are shown above.

```
<wsdl:operation name="updateUser">
<wsdl:input message="impl:updateUserRequest" name="updateUserRequest">
</wsdl:input>
<wsdl:output message="impl:updateUserResponse" name="updateUserResponse">
</wsdl:output>
</wsdl:operation>
```

updateUser operation input and message elements are shown above.

<wsdl:operation name="getUserById">

<wsdl:input message="impl:getUserByIdRequest" name="getUserByIdRequest">

</wsdl: input> <wsdl:output message="impl:getUserByIdResponse" name="getUserByIdResponse">

</wsdl:output>

</wsdl:operation>

getUserById operation input and message elements are shown above.

<wsdl:operation name="insertUser">

<wsdl:input message="impl:insertUserRequest" name="insertUserRequest">

</wsdl:input>

<wsdl:output message="impl:insertUserResponse" name="insertUserResponse">

</wsdl:output>

</wsdl:operation>

insertUser operation input and message elements are shown above.

<wsdl:operation name="deleteUser">

<wsdl:input message="impl:deleteUserRequest" name="deleteUserRequest">

</wsdl:input>

<wsdl:output message="impl:deleteUserResponse" name="deleteUserResponse">

</wsdl:output>

</wsdl:operation>

</wsdl:portType>

deleteUser operation input and message elements are shown above.

Conclusion

The application configuration can be extended to different layers such as services, entities, persistence and data access. The mobile application framework which is neural sketch based can be used for different software applications.

References

[1] Neural sketch learning for conditional program generation

V Murali, L Qi, S Chaudhuri, C Jermaine

[2] Neural attribute machines for program generation

M Amodio, S Chaudhuri, T Reps

[3] Deepcoder: Learning to write programs

M Balog, AL Gaunt, M Brockschmidt, S Nowozin